

Лабораторна робота №1

Тема: Умовні, циклічні конструкції, функції та рекурсія у мові Python.

Мета: Набуття практичних навичок створення програм, що використовують умовні, циклічні конструкції, функції та рекурсію у мові Python.

Лабораторні завдання

Завдання 1. Скласти програму для знаходження значень складної функції, враховуючи її область визначення. Програма повинна виконуватись в окремому вікні, запитувати аргумент, показувати підфункцію, за якою обчислюється значення складної функції. Якщо змінна входить у склад аргументу тригонометричної функції, то (якщо можливо) її значення повинно вводитись у градусах. Якщо значення функції знайти неможливо, програма повинна повідомити про причину цього (наприклад, "Знаменник ... дорівнює нулю" і т. п., де ... – конкретний вираз). Програма повинна математично коректно обробляти всі числові значення аргументу функції, які може ввести користувач:

$$13) \ y = \begin{cases} \operatorname{tg} \frac{x}{1-e^x}, & \text{при } x \leq 0, \\ \sqrt{\ln(x-3)}, & \text{при } x > 1 \end{cases}.$$

Опис коду розробленої програми

Програму реалізовано за допомогою бібліотеки tkinter. Вона зчитує значення x , перевіряє його на коректність і за допомогою конструкції if-elif-else обчислює відповідну гілку складної функції. Застосовано блоки try...except для перехоплення помилок вводу та математичних обмежень (ділення на нуль, корінь/логарифм з від'ємного числа). Аргумент для тригонометричних обчислень автоматично переводиться у радіани.

Приклади роботи програми:

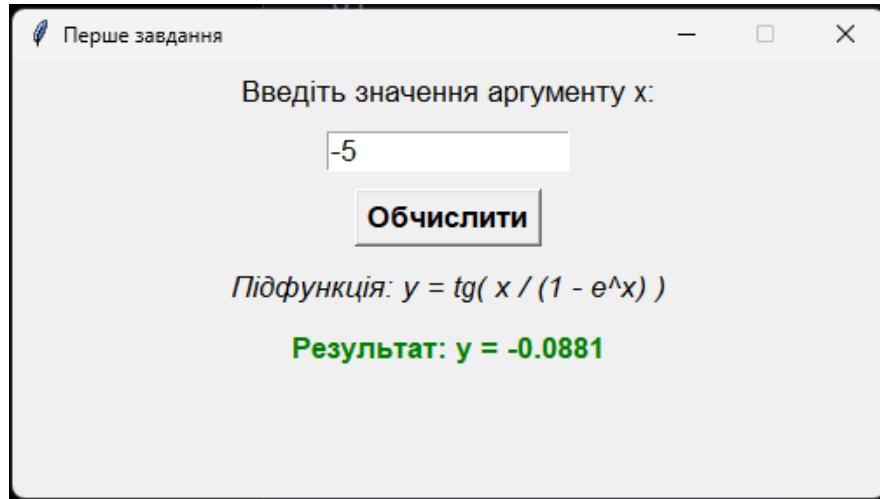


Рис. 1.1 Успішне обчислення першої підфункції

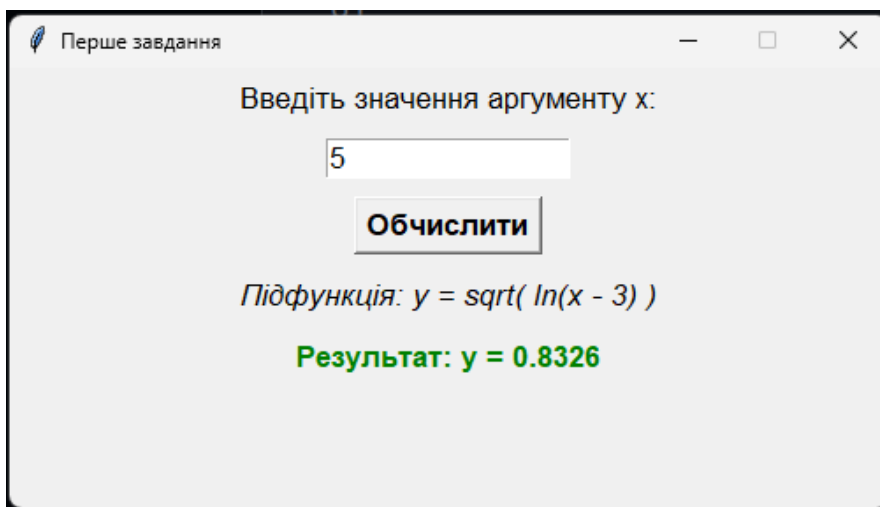


Рис. 1.2 Успішне обчислення другої підфункції

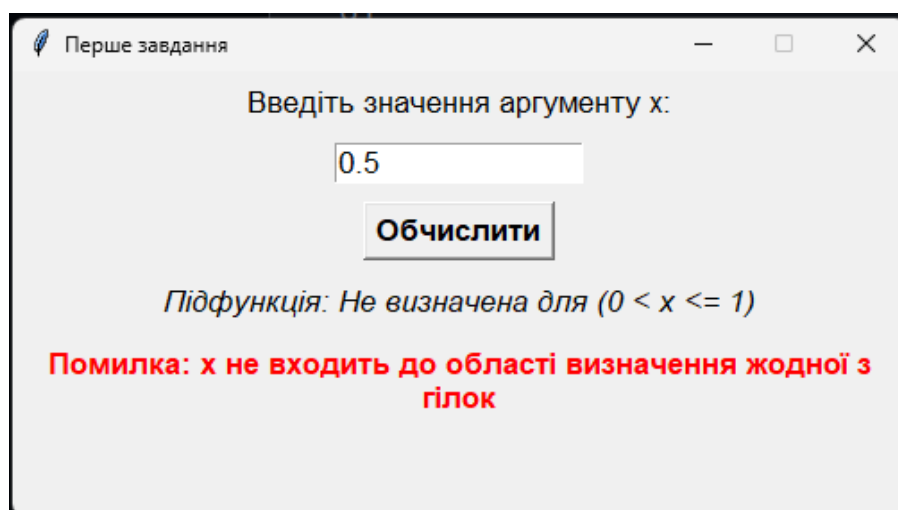


Рис. 1.3 Обробка виняткових ситуацій (область визначення)

Завдання 2. Модифікувати програму знаходження значень складної функції завдання № 1 так, щоб програма працювала, поки користувач не введе деякого "ключового" значення для змінної x . "Ключове" значення повинно задаватися користувачем під час роботи програми.

Опис коду розробленої програми

Програма є доповненням до першого завдання з додаванням функції умовного виходу. До графічного інтерфейсу додано поле для "ключового слова". Перед кожним обчисленням програма порівнює значення, введене в поле аргументу x , із заданим ключовим словом. Якщо вони збігаються, вікно закривається методом `root.destroy()`. В іншому випадку вікно залишається відкритим, продовжуючи цикл обчислень.

Приклад роботи програми:

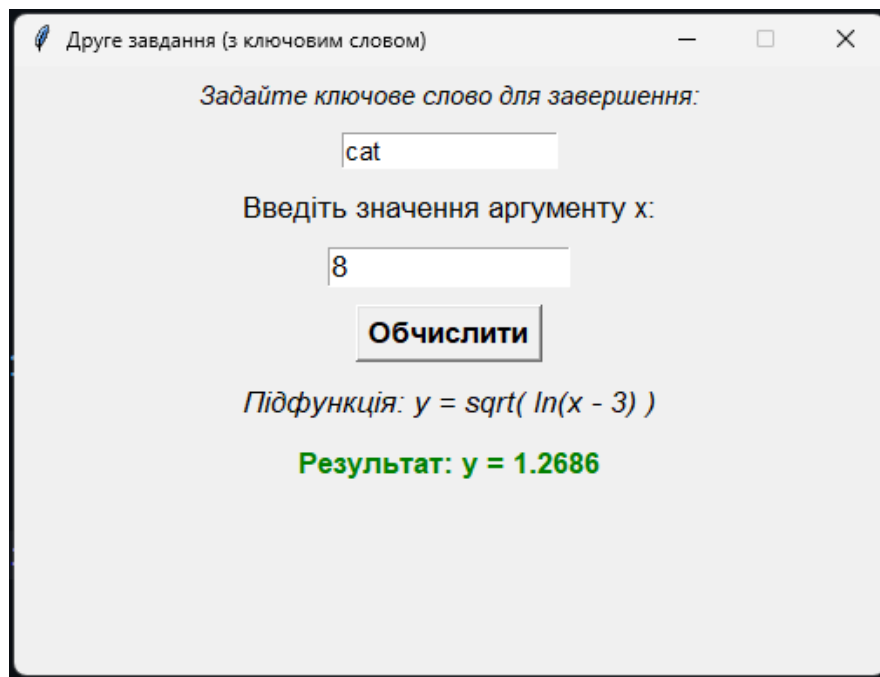


Рис. 2.1 Введення ключового слова для завершення роботи програми

Завдання 3. Скласти програму знаходження значення суми (добутку і т. д.) n елементів виразу. Програма повинна містити 3 функції, перша з яких здійснює зазначене обчислення за допомогою інструкції **for**, друга – **while**, третя – рекурсії. Описані функції повинні повертати логічне значення, яке повідомляє про успішність обчислення, і результат обчислення.

Числа n і x повинні вводитись за допомогою клавіатури. Вважати, що $n \in \mathbb{N}$, $x \in \mathbb{R}$:

$$13) \frac{n-1}{\cos x} + \frac{n-2}{\sin 2x} - \frac{n-3}{\cos 3x} + \frac{n-4}{\sin 4x} + \frac{n-5}{\cos 5x} - \frac{n-6}{\sin 6x} + \dots$$

Опис коду розробленої програми

Консольна програма обчислює суму n елементів математичного ряду трьома способами: за допомогою циклу `for`, циклу `while` та рекурсивної функції. Допоміжна функція `get_term` обчислює значення окремого дробу, враховуючи чергування знаків(++-+-+...) та тригонометричних функцій у знаменнику. Кожен з трьох методів повертає кортеж із двох значень: логічний статус успішності (True/False) та сам результат або текст помилки (наприклад, при діленні на нуль). Введення даних користувачем обробляються перевітками.

Приклади роботи програми:

```
(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova\L1> & C:/Use
• /L1/1_3_13.py
Трете завдання
Введіть кількість елементів n (натуральне число): 3
Введіть значення x (дійсне число, в градусах): 60

Обчислення для n = 3, x = 60.0 градусів:
-----
1. Сума (FOR):      5.154701
2. Сума (WHILE):    5.154701
3. Сума (РЕКУРСІЯ): 5.154701
○ (.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova\L1>
```

Рис. 3.1 – Обчислення суми ряду трьома методами

```
• (.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova\L1> & C:/Users/User/AppData/Local/Python/
/L1/1_3_13.py
Трете завдання
Введіть кількість елементів n (натуральне число): cat
Будь ласка, введіть ціле число

Введіть кількість елементів n (натуральне число): 0,8
Будь ласка, введіть ціле число

Введіть кількість елементів n (натуральне число): 5
Введіть значення x (дійсне число, в градусах): 90

Обчислення для n = 5, x = 90.0 градусів:
-----
1. Сума (FOR):      Помилка: Ділення на нуль (або майже нуль) в елементі i=1
2. Сума (WHILE):    Помилка: Ділення на нуль (або майже нуль) в елементі i=1
3. Сума (РЕКУРСІЯ): Помилка: Ділення на нуль (або майже нуль) в елементі i=1
○ (.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova\L1>
```

Рис. 3.2 – Валідація вводу користувача

Висновки. У ході виконання лабораторної роботи закріплено практичні навички використання базових конструкцій Python. На практиці реалізовано алгоритми з використанням умовних операторів (if-elif-else), циклів (for, while) та рекурсії для обчислення складних математичних функцій і рядів. Розроблено програми з графічним (tkinter) та консольним інтерфейсами, а також успішно застосовано механізми перехоплення винятків (try...except) для захисту програм від некоректного вводу даних та математичних помилок.